Engineering Handbook – Volume 10

AI Developer Guide (Version 1.0)

Purpose

This document defines how future AI coding assistants and software engineers should work on the Energy Management Industrial IoT SaaS platform while preserving architectural integrity.

Core Principles

Never break backward compatibility. Prefer extending existing modules over rewriting them. Keep the platform multi-tenant, modular and configuration-driven.

Coding Standards

Write clean, readable Python. Add comments explaining non-trivial logic. Use type hints, descriptive names and consistent formatting. Avoid duplicate code.

Architecture Rules

Business logic belongs in services, routers should remain thin, schemas define API contracts and models define persistence. Keep responsibilities separated.

Database Changes

Never modify production tables destructively. Use migrations for schema evolution. Preserve historical telemetry and tenant isolation.

API Development

Every new endpoint must include validation, authentication, consistent error handling, documentation and example requests/responses.

Testing

New features should include unit tests where practical. Validate API behavior, authentication, database interactions and edge cases before deployment.

Documentation

Update the engineering handbook and inline code comments whenever architecture or APIs change. Treat documentation as part of the deliverable.

Git Workflow

Develop locally, commit frequently with meaningful messages, push to GitHub and deploy through controlled releases. Never edit production code directly unless performing emergency fixes.

AI Workflow

Before implementing changes, review existing architecture, models, routers and documentation. Explain design decisions. Preserve style consistency and avoid introducing unnecessary dependencies.

Long-Term Vision

The platform should evolve into a generic Industrial IoT operating system supporting multiple industries while keeping textile-specific functionality as optional modules.

AI Checklist

- Read existing documentation first
- Understand database relationships
- Preserve API compatibility
- Document architectural changes
- Keep code modular
- Add comments for complex logic
- Avoid hardcoded values
- Consider scalability and multi-tenancy
- Write production-quality code
- Prefer maintainability over cleverness